Top 20 Git Concepts

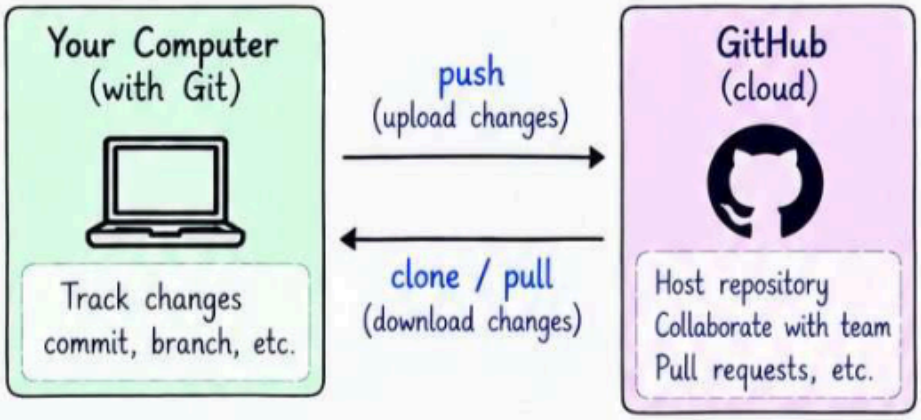
Every Developer Should Know

1 Git vs GitHub

Two different things, often confused

- Git is a version control system that runs on your local machine.
- GitHub is a cloud platform that hosts Git repositories and adds collaboration features.

How it works?



Git vs GitHub

	Git	GitHub
Type	Version control system	Cloud platform
Runs on	Local machine	Cloud (online)
Main purpose	Track changes	Host & collaborate
Internet required	No	Yes

Key Points

- Git is a tool.
- GitHub is a service built on top of Git.

Use Cases

- Use Git to manage code locally.
- Use GitHub to collaborate with others.

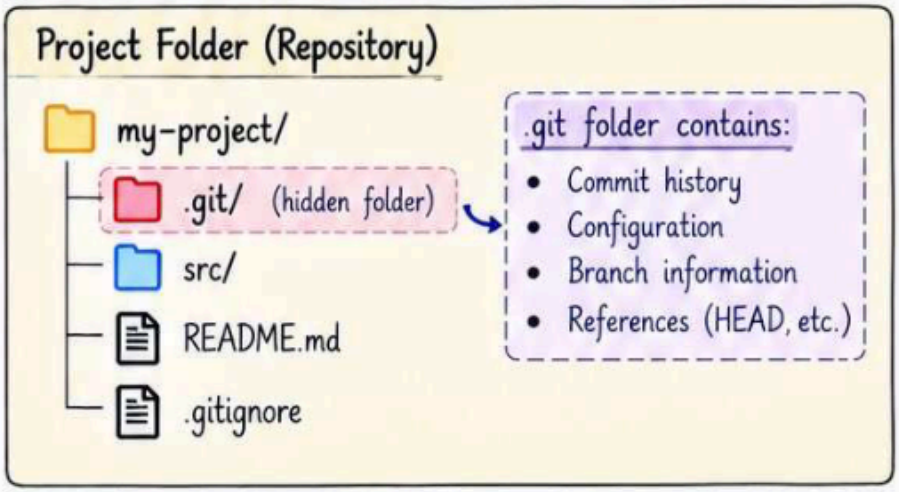
---

2 Repository & Working Directory

Where your code lives

- A repository (repo) is a project folder where Git tracks your files and changes.
- The working directory is the actual folder on your machine where you create, edit and delete files.

How it works?



Key Points

- The .git folder is hidden.
- Git tracks changes in the repository (not just a single file).

Use Cases

- Initialize a new project with a Git repo.
- Manage source code for any application.

Top 20 Git Concepts

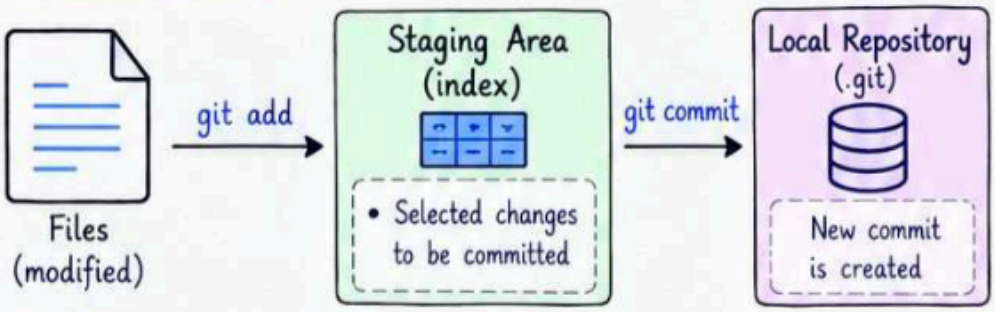
Every Developer Should Know

3 Git Commit

Save changes in Git history

- A commit is a snapshot of your changes at a particular point in time.
- Each commit has a unique ID (hash) and a commit message.

How it works?



Example

```
$ git add file.txt
$ git commit -m "Added file.txt"
```

Commit Created

a1b2c3d (commit hash)
Added file.txt
Author: Aman
Date: Oct 20, 2025

Key Points

- A commit stores only the changes, not the entire files.
- Always write a meaningful commit message.

Use Cases

- Save your work progress.
- Track the history of changes.

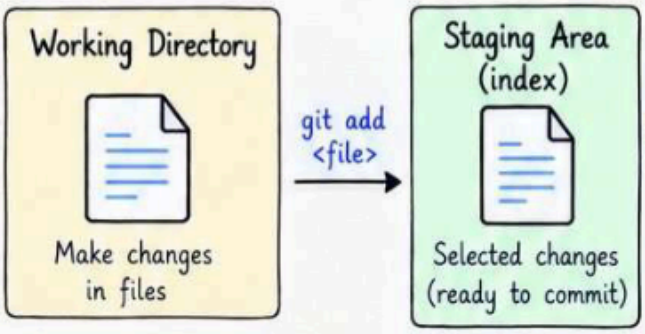
---

4 Staging Area (Index)

Prepare changes before committing

- The staging area (index) is a temporary area where you select changes that you want to include in the next commit.
- You use git add to move changes from your working directory to the staging area.

How it works?



Common Commands

```
$ git add file.txt  # add a file
$ git add .         # add all changes
$ git status        # check status
$ git commit -m "msg"
```

Key Points

- Staging area lets you choose what to commit.
- You can add some files and leave others.

Use Cases

- Commit related changes together.
- Review changes before committing.

Top 20 Git Concepts

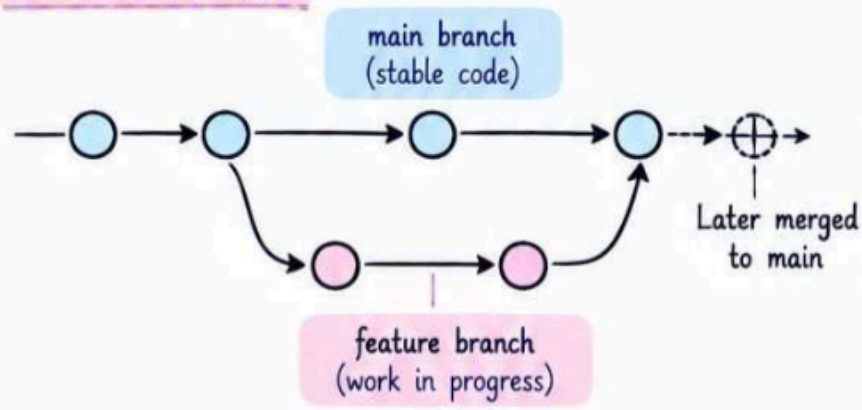
Every Developer Should Know

5 Branches

Work on different features in parallel

- A branch is a separate line of development.
- It allows you to work on new features or fixes without affecting the main code.

How it works?



Common Commands

```
$ git branch                # list all branches
$ git branch feature        # create a new branch
$ git checkout feature      # switch to branch
$ git checkout -b feature   # create and switch
```

Key Points

- Each branch has its own commits.
- Main branch is usually called main or master.

Use Cases

- Developing new features.
- Fixing bugs without affecting the stable code.

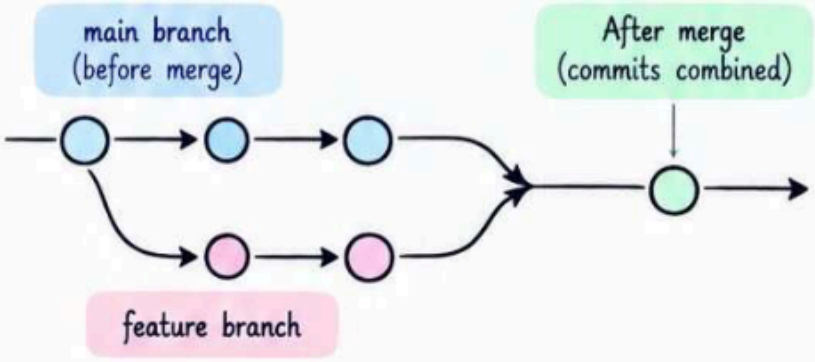
---

6 Merge

Combine changes from one branch into another

- Merge is used to combine the changes from one branch into another branch.
- It brings the commits from the source branch into the target branch.

How it works?



Example

```
$ git checkout main     # switch to target branch
$ git merge feature     # merge feature branch
```

Key Points

- Merge keeps the full history of both branches.
- Can sometimes lead to merge conflicts

Use Cases

- Merging a feature branch into main after review.

Top 20 Git Comcepts

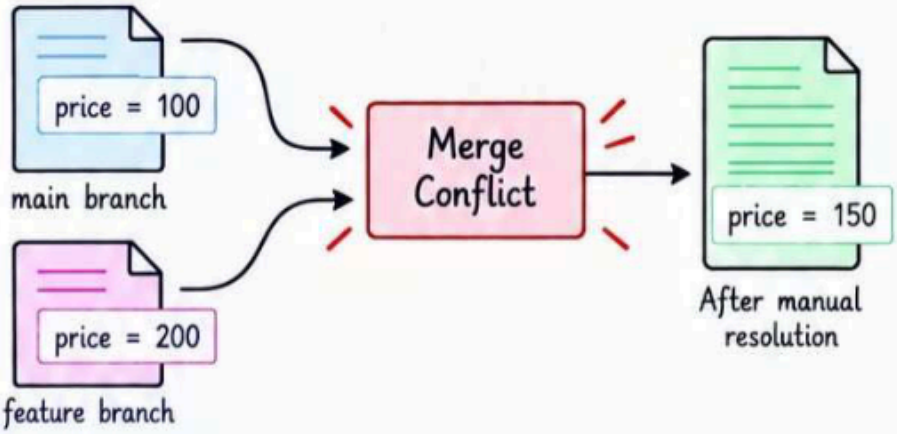
Every Developer Should Know

7 Merge Conflict

When Git can't automatically merge changes

- A merge conflict happens when the same part of a file is changed in different branches.
- Git cannot decide which change to keep, so you have to resolve it manually.

How it works?



Conflict in File

```
<<<<<<< HEAD
price = 100
=======
price = 200
>>>>>>>  feature
```

Key Points

- Conflicts usually happen when the same lines are modified.
- You need to edit the file and choose the correct changes.

Use Cases

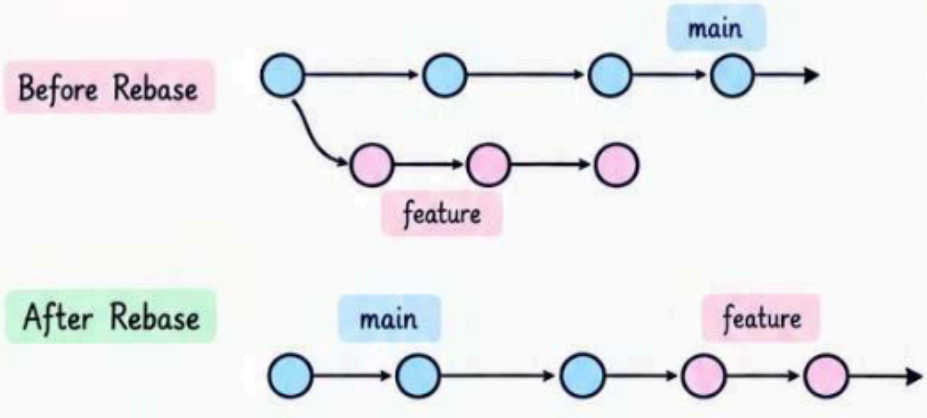
- When multiple developers work on the same file.
- Common in team projects during feature merges.

8 Rebase

Reapply commits on top of another branch

- Rebase moves your commits to the latest commit of another branch.
- It creates a linear history (cleaner commit history).

How it works?



Common Commands

```
# Switch to your feature branch
$ git checkout feature

# Rebase on latest main
$ git rebase main
```

Key Points

Use Cases

Top 20 Git Concepts

Every Developer Should Know

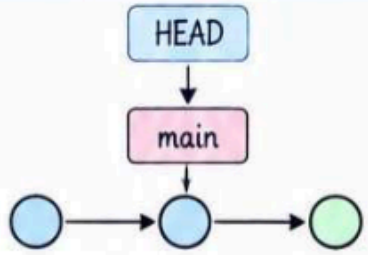
9 HEAD & Detached HEAD

Understand what HEAD points to

- HEAD is a pointer to the current branch (latest commit).
- In detached HEAD state, HEAD points to a specific commit instead of a branch.

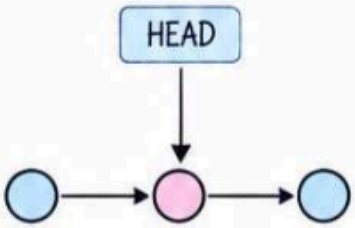
How it works?

Normal HEAD (on a branch)



HEAD points to the latest commit of the current branch.

Detached HEAD (at a commit)



HEAD points to a specific commit (not a branch).

Common Commands

```
# See where HEAD points
$ git branch
$ git log --oneline

# Checkout a specific commit (detached HEAD)
$ git checkout <commit-hash>

# Go back to a branch
$ git checkout main
```

Key Points

- HEAD usually points to the current branch.
- Detached HEAD is useful for viewing old commits but not for long-term work.

Use Cases

- Switch between branches.
- View or test old commits (e.g. for debugging).

---

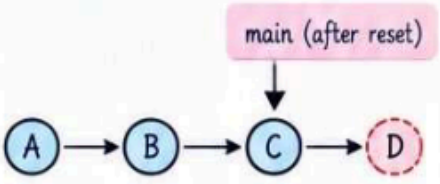
10 Reset vs Revert

Undo changes in different ways

- git reset moves the branch pointer and can remove commits (changes history).
- git revert creates a new commit that undoes previous changes (keeps history).

How it works?

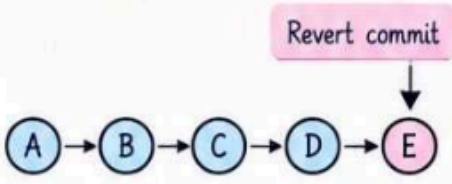
git reset (moves pointer)



Commits after the pointer are removed (from branch history).

Types: --soft, --mixed, --hard

git revert (adds new commit)



A new commit is created that undoes the changes.

Common Commands

```
# Reset (move to previous commit)
$ git reset --hard HEAD~1

# Revert (create new commit)
$ git revert <commit-hash>
```

Key Points

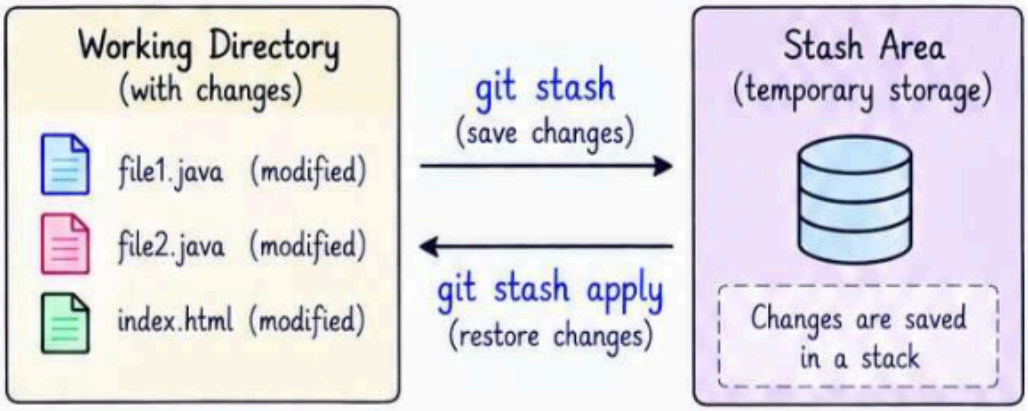
Use Cases

11 Git Stash

Temporarily save your unfinished changes

- Git stash lets you temporarily save changes without committing them.
- Useful when you want to switch branches but don't want to commit incomplete work.

How it works?



Common Commands

```
# Save changes
$ git stash

# See saved stashes
$ git stash list

# Restore changes
$ git stash apply

# Remove the latest stash
$ git stash drop
```

Key Points

- Stash saves uncommitted changes.
- Stashes are stored in a stack.

Use Cases

- Switch branches quickly.
- Work on something else and come back later.

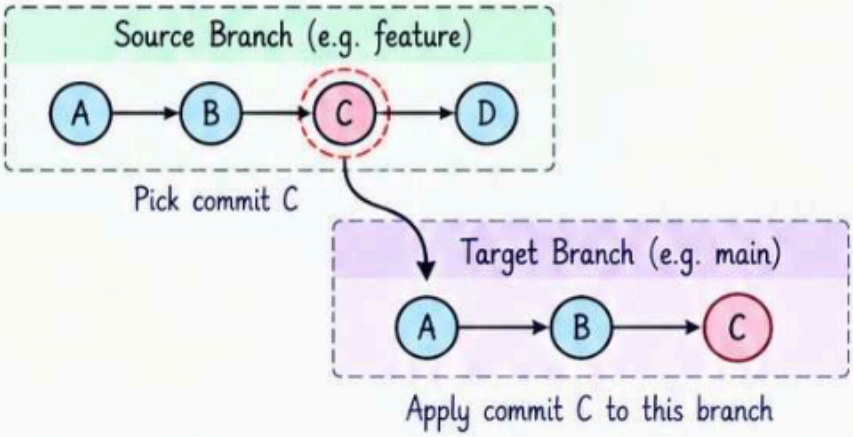
---

12 Cherry-Pick

Bring a specific commit to another branch

- Cherry-pick lets you apply a single commit from one branch to another.
- Useful when you want a specific change without merging the entire branch.

How it works?



Common Commands

```
# Get commit hash (from source branch)
$ git log

# Cherry-pick the commit
$ git cherry-pick <commit-hash>
```

Key Points

- Applies only the selected commit.
- Does not merge the entire branch.

Use Cases

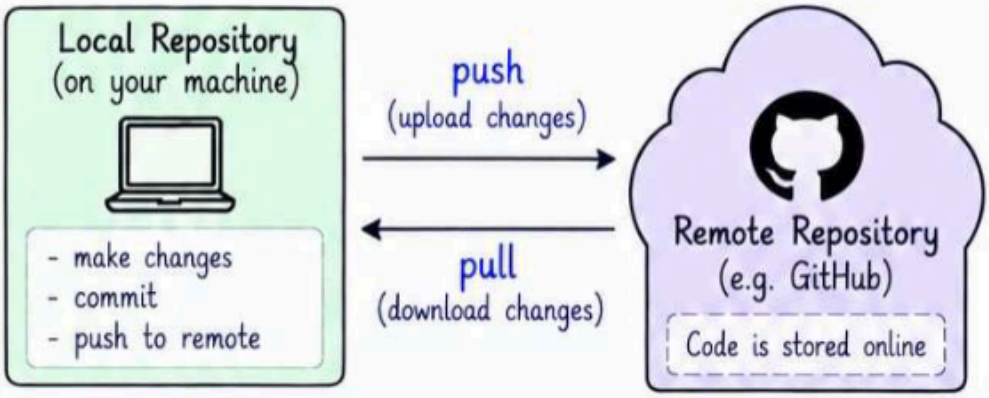
- Apply a bug fix from another branch.
- Pick a specific feature or change.

13 Remote Repository

Store your code in the cloud

- A remote repository is the version of your project hosted on a server (e.g. GitHub, GitLab, Bitbucket).
- It helps you backup your code and collaborate with others.

How it works?



Common Commands

```
# Add a remote repository
$ git remote add origin <url>

# Push your code
$ git push -u origin main

# Pull latest changes
$ git pull origin main
```

Key Points

- Remote repo is hosted on a server (cloud).
- You can have multiple remotes (e.g. origin, upstream).

Use Cases

- Backup your code.
- Collaborate with team members.

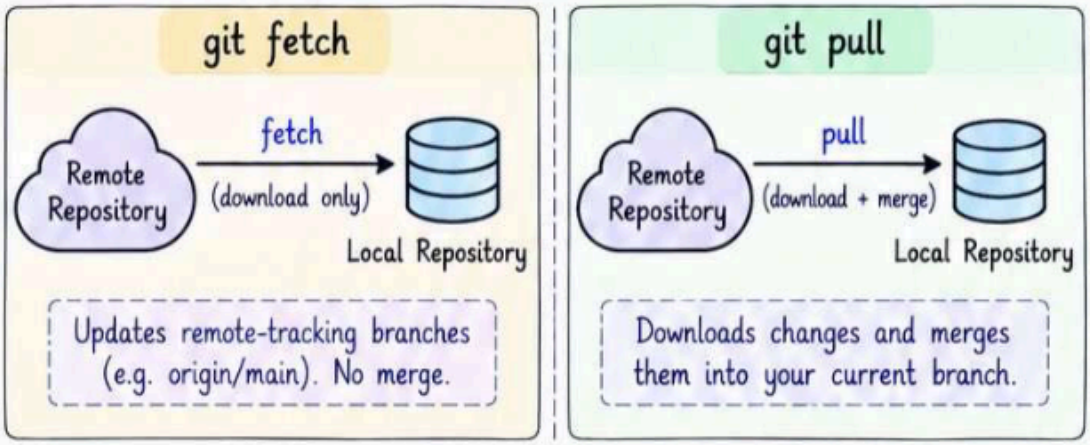
---

14 Fetch vs Pull

Both download changes, but work differently

- git fetch downloads the latest changes from remote but does not merge them.
- git pull downloads the latest changes and merges them into your current branch.

How it works?



Common Commands

```
# Fetch latest changes
$ git fetch origin

# See what's new
$ git branch -r

# Pull (fetch + merge)
$ git pull origin main
```

Key Points

- Fetch is safe (no changes to your code).
- Pull fetches and merges changes.

Use Cases

- Use fetch to review changes before merging.
- Use pull when you want latest changes in your branch.

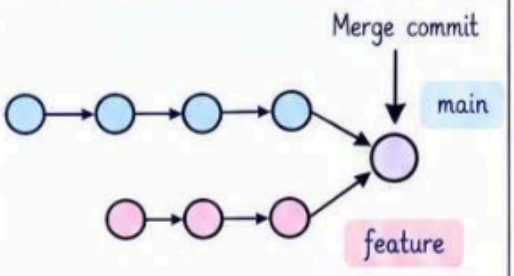
17 Merge vs Rebase

Two ways to integrate changes from one branch to another

- Merge combines the changes and keeps the full history (creates a merge commit).
- Rebase replays your commits on top of another branch (cleaner history).

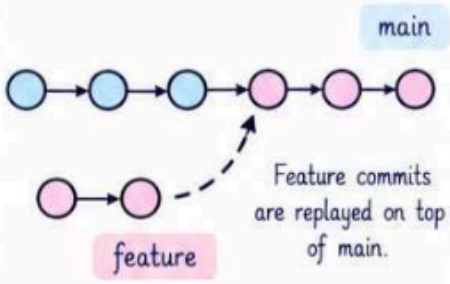
How it works?

Merge (keeps history)



Combines the histories and creates a new merge commit.

Rebase (linear history)



Common Commands

```
# Merge a branch
$ git checkout main
$ git merge feature

# Rebase a branch
$ git checkout feature
$ git rebase main
```

Key Points

- Merge keeps the complete history (good for teams).
- Rebase creates a linear history (cleaner but rewrites commits).

Use Cases

- Use merge for shared branches (e.g. main).
- Use rebase for your local feature branch before opening a PR.

---

18 Merge Conflicts

When Git cannot automatically merge changes

- A merge conflict happens when the same line of code is changed in both branches.
- You need to manually resolve the conflict and then commit the changes.

Example

Conflict in a file (example)

```
1  <<<<<<< HEAD
2  System.out.println("Hello");
3  =========
4  System.out.println("Hi");
5  >>>>>>>> feature
```

Git shows conflict markers. You need to choose or combine the correct changes.

Steps to resolve

1. Open the file with conflicts.
2. Edit the file and keep the correct changes.
3. Remove the conflict markers (<<<<<<<, =======, >>>>>>>).
4. Stage the file.
5. Commit the changes.

Common Commands

```
# Check conflicted files
$ git status

# After resolving conflicts
$ git add <file>
$ git commit -m "Resolve
  merge conflict"
```

Key Points

- Conflicts commonly happen in the same file/line.
- Git cannot decide the correct changes automatically.

Use Cases

- Happens during merge or rebase.
- Common in team environments when multiple developers work on the same code.

Top 20 Git Concepts

Every Developer Should Know

19 Git Workflow

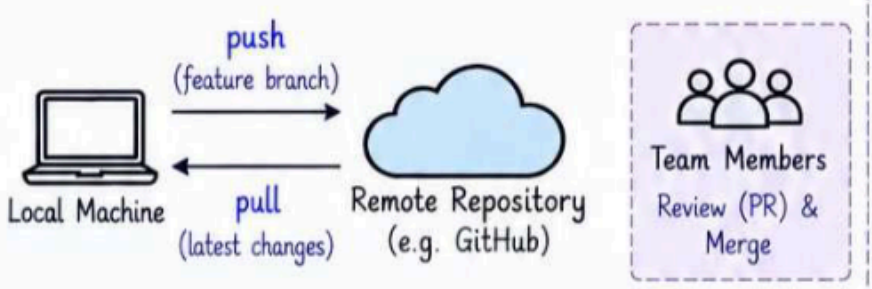
A typical workflow from start to finish

- Git workflow is the set of steps you follow to develop a feature using Git.
- It helps you keep your work organized and collaborate with your team.

Typical Workflow

1. **Clone** — Get the latest code from remote
2. **Create Branch** — Work on a new feature/bug
3. **Make Changes** — Edit files and commit
4. **Push** — Push your branch to remote
5. **Create PR** — Open a Pull Request for review
6. **Merge** — After approval, merge into main

Diagram



Common Commands

```
# Clone the repository
$ git clone <url>

# Create and switch to a new branch
$ git checkout -b feature

# Add, commit and push changes
$ git add .
$ git commit -m "Your message"
$ git push origin feature

# Create a Pull Request (on GitHub/GitLab)
# After review, merge the PR
```

Key Points

- Always pull latest changes before starting work.
- Use a new branch for each feature or bug fix.
- Write clear and meaningful commit messages.
- Raise a Pull Request for code review.
- Keep your branch updated with main to avoid conflicts.

Best Practices

- Use meaningful branch names (e.g. feature/login).
- Keep commits small and focused.
- Pull frequently to stay in sync.
- Use PR reviews to maintain code quality.
- Delete merged branches to keep the repo clean.

20 .gitignore

Exclude unnecessary files from Git

- .gitignore is a file that tells Git which files or folders to ignore (not track).
- Useful for ignoring temporary files, dependencies, logs, etc.

Example .gitignore

```
# Compiled files
*.class
*.log

# IDE folders
.idea/
.vscode/

# Build folders
```

Key Points

- Ignored files are not tracked by Git.
- Good for excluding sensitive, temporary or large files.
- Each project can have its own .gitignore file.

Use Cases

- Ignore build folders (e.g. target/).
- Ignore IDE files (.idea/, .vscode/).
- Ignore log files (*.log).
- Ignore environment files (.env).
- Ignore OS specific files (.DS_Store).